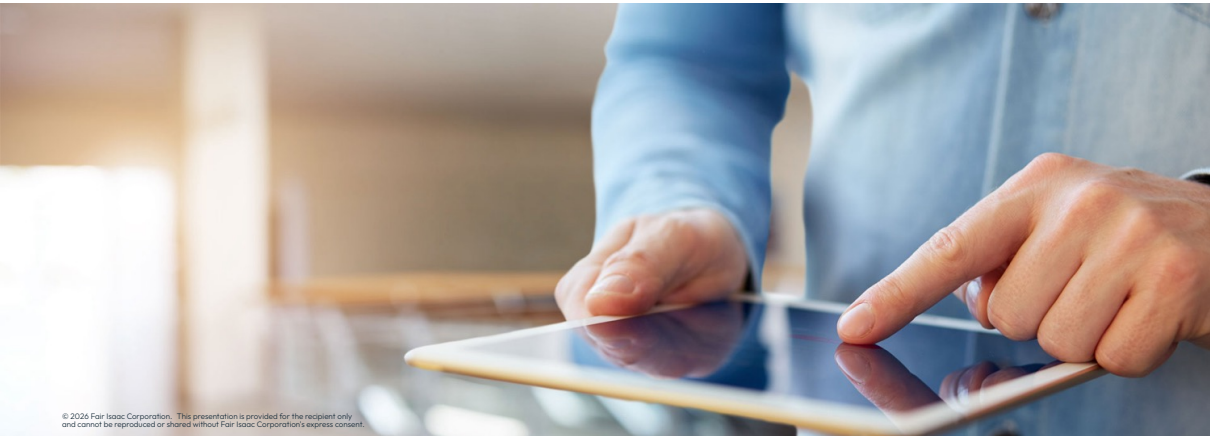
Using Pandas dataframes



Using *Pandas* dataframes with Xpress

Introduction to *Pandas* integration

- The *Pandas* library is a powerful tool for data manipulation and analysis in Python:
  - Provides data structures like DataFrames and Series for handling tabular data
  - Xpress offers enhanced Pandas compatibility for seamless integration with optimization models

Introduction to *Pandas* integration

- The *Pandas* library is a powerful tool for data manipulation and analysis in Python:
  - Provides data structures like DataFrames and Series for handling tabular data
  - Xpress offers enhanced Pandas compatibility for seamless integration with optimization models

- Key benefits of using *Pandas* with Xpress:
  - Efficiently load and manipulate large datasets from CSV, Excel, databases, *etc.*
  - Use familiar Pandas operations (`groupby`, `sum`, `mul`) to create optimization expressions
  - Store Xpress variables directly in DataFrame columns for easy access and manipulation

Hint: *Pandas DataFrames are ideal for optimization problems with structured, tabular input data*

Loading data with *Pandas*

- Loading data from CSV files into a Pandas DataFrame:

```
import pandas as pd
import xpress as xp

# Load stock data from CSV
shares_df = pd.read_csv("data/shares100.csv")
```

Loading data with *Pandas*

- Loading data from CSV files into a Pandas DataFrame:

```
import pandas as pd
import xpress as xp

# Load stock data from CSV
shares_df = pd.read_csv("data/shares100.csv")
```

- The resulting DataFrame contains columns with problem data:

	Stock	Return	Sector	ESG score	CV
0	Stock_1	0.1598	Healthcare	73.0	0.5393
1	Stock_2	0.2018	Technology	95.0	0.6427
2	Stock_3	0.0883	Consumer Goods	63.0	0.2355
3	Stock_4	0.2048	Finance	68.0	0.3360

- Each row represents an entry (e.g., stock, product, facility) and columns contain attributes

Adding variables to DataFrames

- To use Pandas operations with Xpress variables, set the column dtype to 'xpressobj':

```
p = xp.problem("Portfolio Selection")

# Add continuous variables to DataFrame
shares_df['frac'] = pd.Series(
    p.addVariables(len(shares_df), vartype=xp.continuous, name='frac'),
    dtype='xpressobj'
)

# Add binary variables to DataFrame
shares_df['buy'] = pd.Series(
    p.addVariables(len(shares_df), vartype=xp.binary, name='buy'),
    dtype='xpressobj'
)
```

Hint: *The* `dtype='xpressobj'` *is essential for Pandas to correctly handle Xpress variables and expressions*

Building expressions with Pandas operations

- Use element-wise operations to create expressions:
  $\max \sum_{i \in S} RET_i \cdot frac_i$

```
# Objective: maximize expected returns
obj = (shares_df['Return'] * shares_df['frac']).sum()
p.setObjective(obj, sense=xp.ObjSense.MAXIMIZE)
```

Building expressions with Pandas operations

- Use element-wise operations to create expressions:
  $\max \sum_{i \in S} RET_i \cdot frac_i$

```
# Objective: maximize expected returns
obj = (shares_df['Return'] * shares_df['frac']).sum()
p.setObjective(obj, sense=xp.ObjSense.MAXIMIZE)
```

- Pandas operations seamlessly combine data columns with variable columns:
  - `shares_df['Return'] * shares_df['frac']` creates a series of element-wise products
  - `.sum()` aggregates the series into a single Xpress expression

Building expressions with Pandas operations

- Use element-wise operations to create expressions:
  $\max \sum_{i \in S} RET_i \cdot frac_i$

```
# Objective: maximize expected returns
obj = (shares_df['Return'] * shares_df['frac']).sum()
p.setObjective(obj, sense=xp.ObjSense.MAXIMIZE)
```

- Pandas operations seamlessly combine data columns with variable columns:
  - `shares_df['Return'] * shares_df['frac']` creates a series of element-wise products
  - `.sum()` aggregates the series into a single Xpress expression

- Simple constraints using column summation:
  $\sum_{i \in S} frac_i = 1$
  $\sum_{i \in S} buy_i \geqslant$ MinNumStocks

```
# Spend all capital and Minimum number of stocks to purchase
p.addConstraint(shares_df['frac'].sum() == 1)
p.addConstraint(shares_df['buy'].sum() >= MinNumStocks)
```

Element-wise operations

- Pandas supports element-wise arithmetic operations on DataFrame columns:
  $\text{MinPerShare} \cdot buy_i \leqslant frac_i \leqslant \text{MaxPerShare} \cdot buy_i \quad \forall i \in \mathcal{S}$

```
# Linking constraints: if buy[i]=1, then MinPerShare <= frac[i] <= MaxPerShare
p.addConstraint(shares_df['frac'] >= MinPerShare * shares_df['buy'])
p.addConstraint(shares_df['frac'] <= shares_df['buy'].mul(MaxPerShare))
```

Element-wise operations

- Pandas supports element-wise arithmetic operations on DataFrame columns:
  $\text{MinPerShare} \cdot buy_i \leqslant frac_i \leqslant \text{MaxPerShare} \cdot buy_i \quad \forall i \in \mathcal{S}$

```
# Linking constraints: if buy[i]=1, then MinPerShare <= frac[i] <= MaxPerShare
p.addConstraint(shares_df['frac'] >= MinPerShare * shares_df['buy'])
p.addConstraint(shares_df['frac'] <= shares_df['buy'].mul(MaxPerShare))
```

- Two equivalent ways to multiply a column by a scalar:
  - Using the `*` operator: `MinPerShare * shares_df['buy']`
  - Using the `.mul()` method: `shares_df['buy'].mul(MaxPerShare)`

Element-wise operations

- Pandas supports element-wise arithmetic operations on DataFrame columns:
  $\text{MinPerShare} \cdot buy_i \leqslant frac_i \leqslant \text{MaxPerShare} \cdot buy_i \quad \forall i \in \mathcal{S}$

```
# Linking constraints: if buy[i]=1, then MinPerShare <= frac[i] <= MaxPerShare
p.addConstraint(shares_df['frac'] >= MinPerShare * shares_df['buy'])
p.addConstraint(shares_df['frac'] <= shares_df['buy'].mul(MaxPerShare))
```

- Two equivalent ways to multiply a column by a scalar:
  - Using the `*` operator: `MinPerShare * shares_df['buy']`
  - Using the `.mul()` method: `shares_df['buy'].mul(MaxPerShare)`

- Weighted constraint expressions:
  $\sum_{i \in \mathcal{S}} \text{ESG}_i \cdot frac_i \geqslant \text{MinESG}$

```
# Average ESG score must be at least MinESG
avg_esg = (shares_df['ESG score'] * shares_df['frac']).sum() >= MinESG
p.addConstraint(avg_esg)
```

Using `groupby` for aggregate constraints

- The `groupby()` method creates constraints for each group in the data:
  $\sum_{i \in \mathcal{S}: Sector[i]=n} frac_i \leqslant \text{MaxPerSector}, \forall n \in SECTORS$

```
# Maximum investment per sector constraint
p.addConstraint(
    shares_df.groupby('Sector')['frac'].sum() <= MaxPerSector
)
```

Using `groupby` for aggregate constraints

- The `groupby()` method creates constraints for each group in the data:
  $\sum_{i \in \mathcal{S}: Sector[i]=n} frac_i \leqslant \text{MaxPerSector}, \forall n \in SECTORS$

```
# Maximum investment per sector constraint
p.addConstraint(
    shares_df.groupby('Sector')['frac'].sum() <= MaxPerSector
)
```

- How it works:
  - `groupby('Sector')` groups rows by the `Sector` column (e.g., Technology, Healthcare)
  - `['frac']` selects the `frac` variable column within each group
  - `.sum()` sums variables within each group, creating one constraint per sector
  - Note: `groupby()` can also work with multiple columns, e.g., `groupby(['Sector', 'Region'])`

Using `groupby` for aggregate constraints

- The `groupby()` method creates constraints for each group in the data:
  $\sum_{i \in \mathcal{S}: Sector[i]=n} frac_i \leqslant \text{MaxPerSector}, \forall n \in SECTORS$

```
# Maximum investment per sector constraint
p.addConstraint(
    shares_df.groupby('Sector')['frac'].sum() <= MaxPerSector
)
```

- How it works:
  - `groupby('Sector')` groups rows by the `Sector` column (e.g., Technology, Healthcare)
  - `['frac']` selects the `frac` variable column within each group
  - `.sum()` sums variables within each group, creating one constraint per sector
  - Note: `groupby()` can also work with multiple columns, e.g., `groupby(['Sector', 'Region'])`

- This single line creates multiple constraints, one for each unique sector:
  - Much more concise than writing loops or individual constraints
  - Automatically handles groups without needing to enumerate them explicitly

Retrieving and analyzing solutions

- After solving, retrieve variable values back into the DataFrame:

```
p.optimize()

# Get solution values for all variables in the 'frac' column
shares_df["sol_frac"] = p.getSolution(shares_df['frac'])
```

Retrieving and analyzing solutions

- After solving, retrieve variable values back into the DataFrame:

```
p.optimize()

# Get solution values for all variables in the 'frac' column
shares_df["sol_frac"] = p.getSolution(shares_df['frac'])
```

- Use Pandas operations to compute solution metrics:

```
# Compute weighted averages and other metrics
SummaryValues = pd.Series({
    "Expected return": (shares_df["Return"] * shares_df["sol_frac"]).sum(),
    "Average risk": (shares_df["CV"] * shares_df["sol_frac"]).sum(),
    "Average ESG": (shares_df["ESG score"] * shares_df["sol_frac"]).sum(),
    "# selected": (shares_df["sol_frac"] > 0).sum(),
    "Largest position": shares_df["sol_frac"].max(),
})
print(SummaryValues)
```

Filtering and visualization

- Use Pandas filtering to analyze specific solution components:

```
# Filter rows where fraction is at least 0.5%
selected = shares_df[shares_df["fraction"] >= 0.005]

# Sort by fraction in descending order
selected = selected.sort_values('fraction', ascending=False)

print(selected[['Stock', 'Sector', 'Return', 'fraction']])
```

Filtering and visualization

- Use Pandas filtering to analyze specific solution components:

```
# Filter rows where fraction is at least 0.5%
selected = shares_df[shares_df["fraction"] >= 0.005]

# Sort by fraction in descending order
selected = selected.sort_values('fraction', ascending=False)

print(selected[['Stock', 'Sector', 'Return', 'fraction']])
```

- Integration with visualization libraries (*matplotlib*, *seaborn*):

```
import matplotlib.pyplot as plt

# Create pie chart of portfolio composition
plt.pie(selected['fraction'], labels=selected['Stock'])
plt.title('Portfolio Composition')
plt.show()
```

Example: Portfolio optimization problem

- Problem: Select stocks to maximize returns subject to constraints:
  - Investment per stock: $1\% \leqslant \text{frac}_s \leqslant 20\%$ (if selected)
  - Investment per sector $\leqslant 25\%$
  - Minimum 10 different stocks
  - Weighted ESG score $\geqslant 70$
  - Weighted risk (CV) $\leqslant 0.5$
  - Data: 100 stocks with attributes (Return, Sector, ESG score, CV)

Example: Portfolio optimization problem

- Problem: Select stocks to maximize returns subject to constraints:
  - Investment per stock: $1\% \leqslant \text{frac}_s \leqslant 20\%$ (if selected)
  - Investment per sector $\leqslant 25\%$
  - Minimum 10 different stocks
  - Weighted ESG score $\geqslant 70$
  - Weighted risk (CV) $\leqslant 0.5$
  - Data: 100 stocks with attributes (Return, Sector, ESG score, CV)

- Full implementation uses:
  - `pd.read_csv()` to load data
  - DataFrame columns for variables with `dtype='xpressobj'`
  - Element-wise operations for constraints
  - `groupby()` for sector constraints
  - Pandas aggregation for solution analysis

Hint: *See the Jupyter notebook portfolio_pandas.ipynb for the complete implementation*

Thank You

www.fico.com/optimization